Java Kontrollstrukturen

Autor

Luis Staudt

30. November 2025

Inhaltsverzeichnis

Inhaltsverzeichnis 2
Kontrollstrukturen in Java 3
- If-Bedingungen 3
- For-Schleifen 9
- While-Schleifen 17
- Vergleich: For vs. While 20
- Häufige Fehler 21

Kontrollstrukturen in Java

If-Bedingungen

Warum braucht man If-Bedingungen?

If-Bedingungen ermöglichen es, Code nur unter bestimmten Voraussetzungen auszuführen. Sie sind notwendig, wenn das Programm Entscheidungen treffen soll: "Wenn eine Bedingung erfüllt ist, dann führe diese Anweisungen aus."

Typische Anwendungsfälle:

- Prüfen, ob ein Wert einen bestimmten Zustand hat
- Vergleichen von Werten
- Reagieren auf Benutzereingaben
- Fehlerbehandlung (z.B. Index-Prüfung)

Einfache If-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahl pruefen
int alter = 18;
if (alter >= 18) {
    System.out.println("Volljaehrig");
}

// Array-Index pruefen
int[] zahlen = {10, 20, 30};
int index = 1;
if (index >= 0 && index < zahlen.length) {
    System.out.println(zahlen[index]);  // Sicher zugreifen
}

// Zeichen pruefen
String text = "Hallo";
if (text.charAt(0) == 'H') {
    System.out.println("Beginnt mit H");
}

// String-Laenge pruefen
String wort = "Test";
if (wort.length() > 3) {
    System.out.println("Wort ist laenger als 3 Zeichen");
}

// Wert gefunden
boolean gefunden = false;
int[] liste = {5, 10, 15, 20};
int gesucht = 15;
for (int i = 0; i < liste.length; i++) {
    if (liste[i] == gesucht) {
        gefunden = true;
    }
}
```

If-Else-Bedingung

Syntax:

```
if (Bedingung) {
    Anweisung1;
} else {
    Anweisung2;
}
```

Beispiele:

```
// Gerade oder ungerade
int zahl = 7;
if (zahl % 2 == 0) {
    System.out.println("Gerade");
} else {
    System.out.println("Ungerade");
}

// Positiv oder negativ
int wert = -5;
if (wert >= 0) {
    System.out.println("Positiv oder Null");
} else {
    System.out.println("Negativ");
}

// Array leer oder gefuellt
int[] zahlen = {1, 2, 3};
if (zahlen.length > 0) {
    System.out.println("Array hat " + zahlen.length + " Elemente");
} else {
    System.out.println("Array ist leer");
}

// Zeichen vorhanden
String text = "Java";
if (text.length() > 0) {
    char erstes = text.charAt(0);
    System.out.println("Erstes Zeichen: " + erstes);
} else {
    System.out.println("String ist leer");
}
```

If-Else-If-Bedingung

Syntax:

```
if (Bedingung1) {
    Anweisung1;
} else if (Bedingung2) {
    Anweisung2;
} else {
    Anweisung3;
}
```

Beispiele:

```
// Notenbewertung
int punkte = 85;
if (punkte >= 90) {
    System.out.println("Note: Sehr gut");
} else if (punkte >= 80) {
    System.out.println("Note: Gut");
} else if (punkte >= 70) {
    System.out.println("Note: Befriedigend");
} else {
    System.out.println("Note: Nicht bestanden");
}

// Groessenvergleich
int a = 5;
int b = 10;
if (a > b) {
    System.out.println("a ist groesser");
} else if (a < b) {
    System.out.println("b ist groesser");
} else {
    System.out.println("a und b sind gleich");
}

// String-Laenge kategorisieren
String eingabe = "Hallo";
if (eingabe.length() == 0) {
    System.out.println("Leer");
} else if (eingabe.length() < 5) {
    System.out.println("Kurz");
} else if (eingabe.length() < 10) {
    System.out.println("Mittel");
} else {
```

```
33     System.out.println("Lang");
34 }
35
36 // Index-Position bestimmen
37 int[] werte = {10, 20, 30, 40, 50};
38 int index = 2;
39 if (index < 0) {
40     System.out.println("Index zu klein");
41 } else if (index >= werte.length) {
42     System.out.println("Index zu gross");
43 } else {
44     System.out.println("Wert: " + werte[index]);
45 }
```

Verschachtelte If-Bedingungen

Syntax:

```
if (Bedingung1) {
    if (Bedingung2) {
        Anweisung;
    }
}
```

Beispiele:

```
// Doppelte Pruefung
String text = "Test";
if (text.length() > 0) {
    if (text.charAt(0) == 'T') {
        System.out.println("Beginnt mit T und ist nicht leer");
    }
}

// Array-Zugriff mit Pruefung
int[] zahlen = {5, 10, 15, 20};
int index = 2;
if (index >= 0 && index < zahlen.length) {
    if (zahlen[index] > 10) {
        System.out.println("Wert an Position " + index + " ist groesser als
    10");
    }
}

// Mehrere Bedingungen
int zahl = 24;
if (zahl > 0) {
    if (zahl % 2 == 0) {
        if (zahl < 100) {
            System.out.println("Positive gerade Zahl unter 100");
        }
    }
}
```

For-Schleifen

Warum braucht man For-Schleifen?

For-Schleifen ermöglichen es, Code mehrfach auszuführen. Sie sind besonders nützlich, wenn man:

- Alle Elemente eines Arrays durchlaufen möchte
- Alle Zeichen eines Strings prüfen will
- Eine Aufgabe eine bestimmte Anzahl wiederholen möchte
- Mit Indizes arbeiten muss (z.B. zum Ändern von Array-Werten)

Standard For-Schleife

Syntax:

```
for (Initialisierung; Bedingung; Inkrement) {
    Anweisung;
}
```

Wichtig:

Alle drei Teile (Initialisierung, Bedingung, Inkrement) sind **optional** und können weggelassen werden!

Typisches Muster für Arrays:

```
for (int i = 0; i < array.length; i++) {
    // Verwende array[i]
}
```

Typisches Muster für Strings:

```
for (int i = 0; i < text.length(); i++) {
    // Verwende text.charAt(i)
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
for (int i = 0; i < 5; i++) {
    System.out.println(i);  // Gibt aus: 0, 1, 2, 3, 4
}

// Array durchlaufen
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; i < zahlen.length; i++) {
    System.out.println(zahlen[i]);
}

// String durchlaufen
String text = "Hallo";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);
}
```

```java

// Array-Werte aendern
int[] werte = {1, 2, 3, 4, 5};
for (int i = 0; i < werte.length; i++) {
    werte[i] = werte[i] * 2;  // Jeden Wert verdoppeln
}
// Ergebnis: {2, 4, 6, 8, 10}

// Array befuellen
int[] quadrate = new int[5];
for (int i = 0; i < quadrate.length; i++) {
    quadrate[i] = i * i;
}
// Ergebnis: {0, 1, 4, 9, 16}

// Summe berechnen
int[] liste = {5, 10, 15, 20};
int summe = 0;
for (int i = 0; i < liste.length; i++) {
    summe = summe + liste[i];
}
System.out.println("Summe: " + summe);  // Summe: 50

// Zeichen zaehlen
String wort = "Banane";
int anzahlA = 0;
for (int i = 0; i < wort.length(); i++) {
    if (wort.charAt(i) == 'a') {
        anzahlA++;
    }
}
System.out.println("Anzahl 'a': " + anzahlA);  // Anzahl 'a': 3

// Rueckwaerts durchlaufen
int[] zahlen2 = {1, 2, 3, 4, 5};
for (int i = zahlen2.length - 1; i >= 0; i--) {
    System.out.println(zahlen2[i]);  // Gibt aus: 5, 4, 3, 2, 1
}

// Nur gerade Indizes
int[] daten = {10, 20, 30, 40, 50, 60};
for (int i = 0; i < daten.length; i = i + 2) {
    System.out.println(daten[i]);  // Gibt aus: 10, 30, 50
}

// Teilstring durchsuchen (von Position 2 bis Ende)
String satz = "Hallo Welt";
```

```
for (int i = 2; i < satz.length(); i++) {
    System.out.println(satz.charAt(i));  // Gibt aus: l, l, o, , W, e, l, t
}
```

Optionale Teile der For-Schleife

Alle drei Teile können weggelassen werden!

Initialisierung weglassen

Wenn die Variable bereits vorher deklariert wurde:

```
// Beispiel 1: Zaehler bereits vorhanden
int i = 0;
for (; i < 5; i++) {
    System.out.println(i);
}

// Beispiel 2: Start-Index von aussen vorgegeben
int startIndex = 2;
for (; startIndex < 10; startIndex++) {
    System.out.println(startIndex);
}
```

Bedingung weglassen

Wenn die Bedingung immer wahr sein soll (Endlosschleife oder mit `break` beenden):

```
// Beispiel 1: Endlosschleife mit break
int[] zahlen = {10, 20, 30, 40, 50};
for (int i = 0; ; i++) {
    if (i >= zahlen.length) {
        break;  // Schleife manuell beenden
    }
    System.out.println(zahlen[i]);
}

// Beispiel 2: Suche mit vorzeitigem Abbruch
String text = "Hallo Welt";
for (int i = 0; ; i++) {
    if (i >= text.length()) {
        break;
    }
    if (text.charAt(i) == 'W') {
        System.out.println("Gefunden an Position: " + i);
        break;
    }
}
```

Inkrement weglassen

Wenn die Erhöhung manuell im Schleifenkörper erfolgt:

```java
// Beispiel 1: Variable Schrittweite
int[] zahlen = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for (int i = 0; i < zahlen.length; ) {
    System.out.println(zahlen[i]);
    if (zahlen[i] % 2 == 0) {
        i = i + 1;  // Nach geraden Zahlen: 1 Schritt
    } else {
        i = i + 2;  // Nach ungeraden Zahlen: 2 Schritte
    }
}

// Beispiel 2: Bedingte Erhoehung
String text = "aabbbcccc";
for (int i = 0; i < text.length(); ) {
    char c = text.charAt(i);
    System.out.println("Zeichen: " + c);
    // Ueberspringe alle gleichen Zeichen
    while (i < text.length() && text.charAt(i) == c) {
        i++;  // Erhoehung hier statt im for-Kopf
    }
}
```

Mehrere Teile weglassen

```java
// Alle Teile weggelassen = Endlosschleife
// for (;;) {
//     System.out.println("Laeuft unendlich");
// }

// Nur Bedingung vorhanden
int i = 0;
for (; i < 5; ) {
    System.out.println(i);
    i++;
}
```

Verschachtelte For-Schleifen

Syntax:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        Anweisung;
    }
}
```

Beispiele:

```
// Multiplikationstabelle
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.println(i + " * " + j + " = " + (i * j));
    }
}

// Zwei Arrays vergleichen
int[] array1 = {1, 2, 3};
int[] array2 = {2, 3, 4};
for (int i = 0; i < array1.length; i++) {
    for (int j = 0; j < array2.length; j++) {
        if (array1[i] == array2[j]) {
            System.out.println("Gemeinsamer Wert: " + array1[i]);
        }
    }
}

// String-Vergleich (naiv)
String text1 = "abc";
String text2 = "bcd";
for (int i = 0; i < text1.length(); i++) {
    for (int j = 0; j < text2.length(); j++) {
        if (text1.charAt(i) == text2.charAt(j)) {
            System.out.println("Gemeinsames Zeichen: " + text1.charAt(i));
        }
    }
}

// Pattern ausgeben
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
```

```
35     System.out.println();
36 }
37 // Ausgabe:
38 // *
39 // **
40 // ***
41 // ****
```

While-Schleifen

Warum braucht man While-Schleifen?

While-Schleifen verwendet man, wenn:

- Die Anzahl der Wiederholungen im Voraus nicht bekannt ist
- Die Schleife bis zum Eintreten einer bestimmten Bedingung laufen soll
- Man “so lange wie nötig“ wiederholen möchte
- Man nach einem bestimmten Element sucht und abbrechen will, wenn es gefunden wurde

Unterschied zur For-Schleife:

- For-Schleife: “Wiederhole genau n-mal“ oder “Durchlaufe alle Elemente“
- While-Schleife: “Wiederhole solange diese Bedingung wahr ist“

While-Schleife

Syntax:

```
while (Bedingung) {
    Anweisung;
}
```

Beispiele:

```
// Zahlen von 0 bis 4 ausgeben
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}

// Element in Array suchen
int[] zahlen = {10, 20, 30, 40, 50};
int gesucht = 30;
int index = 0;
boolean gefunden = false;
while (index < zahlen.length && !gefunden) {
    if (zahlen[index] == gesucht) {
        gefunden = true;
        System.out.println("Gefunden an Position: " + index);
    }
    index++;
}

// Erste Position eines Zeichens finden
String text = "Hallo Welt";
char gesuchtesZeichen = 'W';
int position = 0;
while (position < text.length()) {
    if (text.charAt(position) == gesuchtesZeichen) {
        System.out.println("Gefunden an Position: " + position);
        break;  // Schleife beenden
    }
    position++;
}

// Solange Wert unter Grenze
int summe = 0;
int zahl = 1;
while (summe < 100) {
```

```
37      summe = summe + zahl;
38      zahl++;
39  }
40  System.out.println("Summe: " + summe);
41  System.out.println("Zahlen addiert: " + (zahl - 1));
42
43  // Array durchlaufen (alternative zu for)
44  int[] werte = {5, 10, 15, 20};
45  int idx = 0;
46  while (idx < werte.length) {
47      System.out.println(werte[idx]);
48      idx++;
49  }
50
51  // Bedingung wird nie erfuellt -> Schleife laeuft nicht
52  int x = 10;
53  while (x < 5) {
54      System.out.println("Wird nicht ausgegeben");
55      x++;
56  }
57
58  // Countdown
59  int countdown = 5;
60  while (countdown > 0) {
61      System.out.println(countdown);
62      countdown--;
63  }
64  System.out.println("Start!");
65
66  // Erstes Vorkommen eines Teilstrings finden
67  String satz = "Der Hund spielt im Garten";
68  String suche = "spielt";
69  int pos = 0;
70  boolean gefundenString = false;
71  while (pos <= satz.length() - suche.length() && !gefundenString) {
72      boolean match = true;
73      for (int j = 0; j < suche.length(); j++) {
74          if (satz.charAt(pos + j) != suche.charAt(j)) {
75              match = false;
76          }
77      }
78      if (match) {
79          gefundenString = true;
80          System.out.println("Gefunden an Position: " + pos);
81      }
82      pos++;
83  }
```

Vergleich: For vs. While

Wann For-Schleife?

```
// Wenn die Anzahl der Durchlaeufe bekannt ist
for (int i = 0; i < 10; i++) { }

// Beim Durchlaufen von Arrays
int[] zahlen = {1, 2, 3, 4, 5};
for (int i = 0; i < zahlen.length; i++) { }

// Beim Durchlaufen von Strings
String text = "Hallo";
for (int i = 0; i < text.length(); i++) { }

// Wenn man den Index braucht
for (int i = 0; i < array.length; i++) {
    array[i] = i * 2;  // Index wird zum Aendern gebraucht
}
```

Wann While-Schleife?

```
// Wenn die Anzahl der Durchlaeufe unbekannt ist
while (summe < 1000) {
    summe = summe + wert;
}

// Bei Suche mit vorzeitigem Abbruch
while (index < array.length && !gefunden) {
    if (array[index] == gesucht) {
        gefunden = true;
    }
    index++;
}

// Bedingungsabhaengige Wiederholung
while (bedingungErfuellt()) {
    macheEtwas();
}
```

Häufige Fehler

Endlosschleifen

```
// FALSCH - Endlosschleife!
int i = 0;
while (i < 5) {
    System.out.println(i);
    // i++ vergessen! Schleife laeuft unendlich
}

// RICHTIG
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;  // Zaehler erhoehen
}
```

Falsche Bedingungen

```
// FALSCH - ArrayIndexOutOfBoundsException!
int[] zahlen = {1, 2, 3};
for (int i = 0; i <= zahlen.length; i++) {  // <= ist falsch!
    System.out.println(zahlen[i]);
}

// RICHTIG
for (int i = 0; i < zahlen.length; i++) {  // < ist richtig
    System.out.println(zahlen[i]);
}
```

Semikolon-Fehler

```
// FALSCH - Semikolon beendet Schleife sofort
for (int i = 0; i < 5; i++);  // Semikolon hier ist falsch!
{
    System.out.println(i);  // Wird nur einmal ausgefuehrt, i nicht bekannt
}

// RICHTIG
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Off-by-One Fehler

```java
String text = "Hallo";  // length = 5, Indizes: 0-4

// FALSCH
for (int i = 0; i <= text.length(); i++) {  // i = 5 ist zu viel!
    System.out.println(text.charAt(i));
}

// RICHTIG
for (int i = 0; i < text.length(); i++) {
    System.out.println(text.charAt(i));
}

// RICHTIG - Letztes Element direkt
char letztes = text.charAt(text.length() - 1);
```